

AutoGraphForge: Conjecturing, Refutation and Lean Proving in One Loop

Ján Pastorek 

Department of Applied Informatics, Comenius University in Bratislava, Slovakia
`jan.pastorek@fmph.uniba.sk`

Abstract. AutoGraphForge couples conjecturing, refuting, formalizing and proving into one loop for graph theory. Counterexample-guided rounds produced 6,522 conjectures that survive an implication-deciding novelty filter over 559 known relations, refutation against a corpus of 348,207 graphs, and six counterexample-search algorithms; survivors are deterministically exported to Lean 4 with Mathlib, where provers attempt them under an independent kernel check.

Keywords: Automated conjecturing · Graph theory · Automated theorem proving · Lean · Counterexample search

Motivation. Conjecturing systems such as GRAFFITI [4] and the recent GRAFFITI³ [3] have developed separately from proof assistants such as Lean 4 [8]. A substantial literature, in turn, develops search algorithms for (counter)examples [6]. We aim to join these automations into one loop. AutoGraphForge is our prototype (code, data and logs at <https://github.com/JanPastorek/AutoGraphForge>). It works in rounds: every counterexample enters the dataset driving the next round.

Generation of conjectures. We use GRAFFITI³ to generate conjectures over a *small* evolving dataset T carrying 59 exactly computed invariants from GRAPHCALC [2]. Crucially T grows *only* by counterexamples to its own conjectures, so generation stays cheap.

Novelty filter as an LP feasibility test. A perennial problem is separating known or trivial conjectures from interesting ones. We curate 559 classical, folklore and trivial relations, closed under transitive composition and linear identity substitution, each tagged with the class on which it holds. A candidate $f \leq R(g)$ —target invariant f , R linear in the remaining invariants g —is redundant if a convex combination of tabled bounds B_j on f dominates it, certified by weights $w_j \geq 0$ summing to 1, multipliers s_i , μ_k and a slack $t \geq 0$ with, identically in g ,

$$R(g) - \sum_j w_j B_j(g) = \sum_i s_i (g_i - L_i) + \sum_k \mu_k E_k(g) + t,$$

for $g_i \geq L_i$ safe lower bounds and $E_k(g) = 0$ class identities (e.g. Gallai’s $n = \alpha + \tau$); matching coefficients reduces this to LP feasibility. “Is this already known?” thus becomes a decidable query.

Refutation, formalization, proving. Survivors are tested against 348,207 graphs: the House of Graphs export [1], all connected graphs on at most nine vertices and selected extremal families, then against several random graph models and six counterexample-search algorithms from [6]. Since a conjecture is already a structured object—named invariants under class predicates—the Lean 4 export is deterministic: a `sorry`-terminated skeleton elaborating to a type-correct goal. The neural prover OPROVER-32B [7] and the ANTIGRAVITY TEAMWORK multi-agent orchestration framework [5] then search for proofs, kernel-checked against pinned `mathlib4` and our preamble. We also plan to explore `native_decide` as a complement where kernel reduction is insufficient.

Results and status. After 5 rounds of generation–refutation, 6,522 conjectures survived. We inspected the 100 survivors that are sharp on the most graphs and, with LLM assistance, tagged 48 as not known to be trivial or false; among these is a pair relating the annihilation number a (the largest k whose k smallest degrees sum to at most the edge count) and the edge-cover number ec that we then proved by hand—bipartite $\Rightarrow ec \leq a$, regular $\Rightarrow a \leq ec$ —via Gallai, Kőnig and Pepper’s $\alpha \leq a$ [9]. So far the proving stage has kernel-verified only trivial bounds, and scaling it to the survivors is our next step.

Acknowledgments. Computations used the PERUN supercomputer (Supercomputing Centre, TUKE, Slovakia), funded by the EU Recovery and Resilience Plan of the Slovak Republic, project No. 17I03-04-P03-00001.

Disclosure of Interests. The author has no competing interests to declare.

References

1. Coolsaet, K., D’hondt, S., Goedgebeur, J.: House of graphs 2.0: a database of interesting graphs and more. *Discrete Applied Mathematics* **325**, 97–107 (2023). <https://doi.org/10.1016/j.dam.2022.10.013>
2. Davila, R.: GraphCalc: A Python package for computing graph invariants in automated conjecturing systems. *Journal of Open Source Software* **10**(112), 8383 (2025). <https://doi.org/10.21105/joss.08383>
3. Davila, R.: Graffiti³: Compact theory libraries for automated mathematical discovery. Preprint (2026). <https://doi.org/10.13140/RG.2.2.20757.18402>
4. Fajtlowicz, S.: On conjectures of Graffiti. *Discrete Mathematics* **72**(1–3), 113–118 (1988)
5. Google: Gemini multi-agent teams in Antigravity. <https://blog.google/innovation-and-ai/technology/developers-tools/antigravity-teamwork-multi-agent/> (2026), blog post, August 31, 2026
6. Jooker, J.: Computer-assisted graph theory: a survey. *arXiv:2508.20825* (2025)
7. M-A-P: OProver: A unified framework for agentic formal theorem proving. *arXiv:2605.17283* (2026)
8. de Moura, L., Kong, S., Avigad, J., Van Doorn, F., von Raumer, J.: The Lean theorem prover (system description). In: CADE. pp. 378–388. Springer (2015)
9. Pepper, R.: Binding Independence. Ph.D. thesis, University of Houston (2004)